Internal Verification

Lecture 20

Section 1

Part I: Two Styles of Verification

What we did last time

In Lecture 19 we wrote programs and then proved theorems about them:

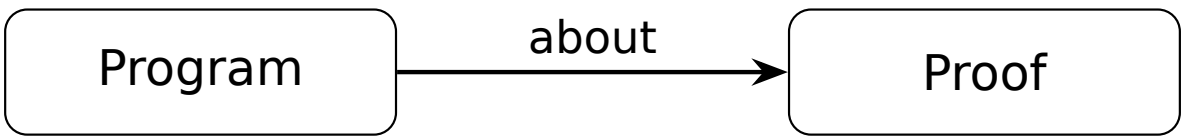
```
def append : List A → List A → List A
  | .nil,       ys ⇒ ys
  | .cons x xs, ys ⇒ .cons x (append xs ys)

theorem append_assoc (xs ys zs : List A)
    : append (append xs ys) zs
    = append xs (append ys zs) := by
  induction xs with ...
```

The program and the proof are **separate**. First we define append, then we prove append_assoc as a separate theorem.

External verification

This style is called **external verification**: write a program, then prove properties about it externally. The proofs are separate artifacts.



Every theorem in Lecture 19 followed this pattern: `add_comm`, `rev_rev`, `length_append`. The function exists first, and the theorem talks *about* it.

Internal verification

There is another style: encode the property **directly in the type** of the program. If the program typechecks, the property holds. No separate theorem needed.

Program *is* the proof

The type is rich enough that the type checker verifies the property during type-checking. The program carries the proof *inside* it.

This is called **internal verification**. It is what dependent types make possible.

The two styles, side by side

Consider the claim: “appending two lists produces a list whose length is the sum of the input lengths.’’

External: Define `append : List A → List A → List A`. Then prove `length (append xs ys) = add (length xs) (length ys)` as a separate theorem.

Internal: Define `vappend : Vec A m → Vec A n → Vec A (add m n)`. The return *type* says the lengths add up. If the function typechecks, the property holds. No theorem needed.

The internal version uses Vec $A\ n$: a list that carries its length in the type. The type checker enforces the length invariant silently.

Why internal verification matters

External proofs can be forgotten. You can write append and ship it without ever proving append_assoc. The program compiles either way.

Internal verification makes this impossible. If the type says Vec A (add m n), the function **must** produce a vector of that length, or it does not typecheck. The guarantee is inseparable from the code.

Today we will see both: vectors (an internally verified list) and binary search trees (an internally verified tree). In both cases, the data structure carries its invariant in the type.

Section 2

Part II: Vectors

Definition

A **vector** is a list indexed by its length:

```
inductive Vec (A : Type) : N → Type where
  | nil  : Vec A .zero
  | cons : A → Vec A n → Vec A (.succ n)
```

A is a **parameter** (the same in every constructor). The natural number is an **index**: nil targets zero, cons targets succ n. The length is encoded in the type, not computed by a function.

Building vectors

```
#check Vec.nil
-- Vec.nil : Vec A .zero

#check Vec.cons 3 Vec.nil
-- Vec.cons 3 Vec.nil : Vec Nat (.succ .zero)

#check Vec.cons 1 (Vec.cons 2 (Vec.cons 3 Vec.nil))
-- : Vec Nat (.succ (.succ (.succ .zero)))
```

The type records the length. You cannot confuse a three-element vector with a two-element vector: they have different types.

Head: total by construction

With List, taking the head requires either an Option wrapper or a partial function. With Vec, we can rule out the empty case in the type:

```
def head : Vec A (.succ n) → A
  | .cons x _ ⇒ x
```

The input has type Vec A (succ n). The only constructor targeting succ is cons. The nil branch is impossible — the type checker verifies this.

No runtime check, no Option, no exception. The type *guarantees* the input is non-empty.

Tail

```
def tail : Vec A (.succ n) → Vec A n
  | .cons _ xs ⇒ xs
```

The return type is Vec $A\ n$: the length decreases by exactly one. The type checker verifies that xs : Vec $A\ n$ matches the required return type.

Compare with `List.tail`: it returns a List A, and you have no static guarantee about the length of the result. With Vec, the length relationship is enforced by the type.

Map

```
def vmap (f : A → B) : Vec A n → Vec B n
  | .nil       ⇒ .nil
  | .cons x xs ⇒ .cons (f x) (vmap f xs)
```

Input and output have the **same index** n: mapping preserves the length. In the nil branch, $n =$ zero and nil : Vec B zero. In the cons branch, $n =$ succ k and the recursive call produces Vec B k, so
cons $(f\ x)$ (vmap f xs) : Vec B (succ k).

This is internal verification: the type says “map preserves length,’’ and the type checker confirms it. No external `length_map` theorem needed.

Zip

```
def vzip : Vec A n → Vec B n → Vec (A × B) n
  | .nil,       .nil       ⇒ .nil
  | .cons x xs, .cons y ys ⇒ .cons (x, y) (vzip xs ys)
```

Both inputs share the index n, so they have the same length. The type checker ensures the match is exhaustive: when the first vector is nil, $n =$ zero, so the second must also be nil. The “mismatched lengths’’ case is ruled out by the types.

With List, zipping two lists of different lengths silently truncates. With Vec, different lengths means different types, and the program will not compile.

The external version of these guarantees

Without vectors, proving “map preserves length’’ requires a separate theorem:

```
theorem length_map (f : A → B) (xs : List A)
    : length (map f xs) = length xs := by
  induction xs with
  | nil         ⇒ rfl
  | cons x xs ih ⇒
    simp only [map, length]; rw [ih]
```

This is five lines of proof for a property that Vec gives us for free. The more invariants you encode in types, the fewer external theorems you need.

Append: where it gets interesting

```
def vappend : Vec A m → Vec A n → Vec A (add m n)
  | .nil,       ys ⇒ ys
  | .cons x xs, ys ⇒ .cons x (vappend xs ys)
```

The return type Vec A (add m n) says the lengths add up. But whether this typechecks depends on how add is defined.

The definitional equality issue

Recall our add from Lecture 19 — it recurs on the **second** argument:

```
def add : N → N → N
  | m, .zero   ⇒ m
  | m, .succ k ⇒ .succ (add m k)
```

In the nil branch of vappend, $m =$ zero, so we need the return type Vec A (add zero n), but ys : Vec A n. We need add zero $n \equiv n$, but the ι-step is stuck (second argument n is a variable, not a constructor). This is not definitional.

The function does **not typecheck** with this definition of add.

Choosing the right recursion

If we define addition to recur on the **first** argument instead:

```
def add' : N → N → N
  | .zero,   n ⇒ n
  | .succ m, n ⇒ .succ (add' m n)
```

Now add′ zero $n \equiv n$ (the ι-step fires). And add′ $(\text{succ}\ m)\ n \equiv \text{succ}\ (\text{add}'\ m\ n)$, which matches the cons branch. With `add'`, vappend typechecks without any casts.

Lesson: in internal verification, the choice of which argument to recur on has *type-level consequences*. The same mathematical function can have different definitional behavior.

The cost of the wrong choice

If you are stuck with the original add, you can enter tactic mode and use `rw` to bridge the gap:

```
def vappend' : Vec A m → Vec A n → Vec A (add m n)
  | .nil, ys ⇒ by
    -- goal: Vec A (add .zero n)
    rw [zero_add]
    -- goal: Vec A n
    exact ys
  | .cons x xs, ys ⇒ by
    -- goal: Vec A (add (.succ _) n)
    rw [succ_add]
    -- goal: Vec A (.succ (add _ n))
    exact .cons x (vappend' xs ys)
```

Each branch enters tactic mode with `by`. The `rw` rewrites the return type using our lemmas from Lecture 19, and then `exact` provides the term.

Replicate

Not every vector function is hard. Here is one that always works:

```
def replicate : (n : N) → A → Vec A n
  | .zero,   _ ⇒ .nil
  | .succ k, a ⇒ .cons a (replicate k a)
```

We recur on n itself. The base case produces Vec A zero; the step produces Vec A $(\text{succ}\ k)$. Both match the return type exactly.

When the recursion structure aligns with the index structure, everything is smooth.

Section 3

Part III: Sigma Types

Dependent pairs

Sometimes we want to *return* a value together with a fact about it. A **sigma type** (dependent pair) bundles a value with a proof:

```
structure DSigma (A : Type) (B : A → Type) where
  fst : A
  snd : B fst
```

The type of the second component *depends on the value of the first*. Lean writes this as $\Sigma\ (a : A),\ B\ a$.

Note: $B : A \to$ Type, not $A \to$ Prop. The second component is **data** (e.g., a vector), not a proof. When the second component is a proof, Lean uses `Subtype`, written $\{a : A\ //\ P\ a\}$.

Example: filtering a vector

Filtering a vector is tricky: we do not know the output length in advance. With a sigma type, we can return the length *and* the vector together:

```
def vfilter (p : A → Bool)
    : Vec A n → DSigma N (Vec A)
  | .nil ⇒ ⟨.zero, .⟩nil
  | .cons x xs ⇒
    let ⟨k, ⟩ys := vfilter p xs
    match p x with
    | true  ⇒ ⟨.succ k, .cons x ⟩ys
    | false ⇒ ⟨k, ⟩ys
```

The return type is DSigma N (Vec A): a natural number k paired with a vector of length k. We do not commit to the output length in the type — we compute it as we go.

Section 4

Part IV: Internally Verified Binary Search Trees

Motivation

Vectors show internal verification for a simple invariant (length). Now we tackle a richer one: the **ordering invariant** of a binary search tree.

A binary search tree (BST) satisfies: for every node with key k, all keys in the left subtree are $\leq k$, and all keys in the right subtree are $\geq k$.

External: define a plain tree, then prove the ordering property separately.

Internal: define a tree type that *can only represent valid BSTs*. An invalid tree cannot even be constructed.

The unverified tree

First, a plain binary tree with no ordering guarantee:

```
inductive Tree (A : Type) : Type where
  | leaf : Tree A
  | node : Tree A → A → Tree A → Tree A
```

Nothing prevents us from building:

```
-- "BST" with 5 on the left of 3: INVALID
#check Tree.node
  (Tree.node Tree.leaf 5 Tree.leaf)
  3
  Tree.leaf
```

The type Tree Nat admits every binary tree, ordered or not.

Comparison on natural numbers

Before building the verified BST, we need comparison. We define a Boolean comparison on our type N:

```
def N.ble : N → N → Bool
  | .zero,   _        ⇒ true
  | .succ _, .zero    ⇒ false
  | .succ m, .succ n  ⇒ N.ble m n
```

This decides $m \leq n$. We will use it to guard the BST invariant.

Bounds

The BST invariant says: every key is between a lower bound and an upper bound. We introduce a type of bounds that includes $-\infty$ and $+\infty$:

```
inductive Bound : Type where
  | bot : Bound    -- ∞-: less than everything
  | val : N → Bound
  | top : Bound    -- ∞+: greater than everything
```

And extend comparison to bounds:

```
def Bound.ble : Bound → Bound → Bool
  | .bot,   _       ⇒ true
  | _,      .top    ⇒ true
  | .val m, .val n  ⇒ N.ble m n
  | _,      _       ⇒ false
```

The verified BST

Now we define a BST **indexed by its bounds**:

```
inductive BST : Bound → Bound → Type where
  | leaf : BST lo hi
  | node : (k : N)
         → Bound.ble lo (.val k) = true
         → Bound.ble (.val k) hi = true
         → BST lo (.val k)
         → BST (.val k) hi
         → BST lo hi
```

Reading the definition

Each constructor of BST carries information:

leaf is a valid BST between any bounds — an empty tree trivially satisfies the ordering.

node $k\ p_1\ p_2\ \mathit{left}\ \mathit{right}$ carries:

- A key $k : N$.
- A proof p_1 that the lower bound is $\leq k$.
- A proof p_2 that $k \leq$ the upper bound.
- A left subtree left : BST lo $(\text{val}\ k)$, bounded above by k.
- A right subtree right : BST $(\text{val}\ k)\ \mathit{hi}$, bounded below by k.

The bounds narrow as you descend. Every node *must* fit within its bounds, or it cannot be constructed.

Why the invalid tree cannot be built

Recall the invalid tree: 5 on the left of 3. To build it as a BST, we would need:

```
BST.node 3
  (sorry)  -- Bound.ble .bot (.val 3) = true  ✓
  (sorry)  -- Bound.ble (.val 3) .top = true   ✓
  (BST.node 5
    (sorry)  -- Bound.ble .bot (.val 5) = true  ✓
    (sorry)  -- NEED: Bound.ble (.val 5) (.val 3) = true
    ...      --  but N.ble 5 3 = false.  ✗
    ...)
  BST.leaf
```

The fourth proof obligation asks for $5 \leq 3$, which is false. No proof exists. The invalid tree **cannot be constructed**.

Searching the BST

```
def search (k : N) : BST lo hi → Bool
  | .leaf ⇒ false
  | .node k' _ _ left right ⇒
    if N.ble k k' then
      if N.ble k' k then true       -- k = k'
      else search k left             -- k < k'
    else search k right               -- k > k'
```

Search does not need the proofs; it only uses the key. But the proofs are there in the data, guaranteeing that the comparisons steer us correctly.

Inserting into the BST

Insertion is where internal verification earns its keep. We need to produce a valid BST with the same bounds:

```
def insert (k : N)
    (p1 : Bound.ble lo (.val k) = true)
    (p2 : Bound.ble (.val k) hi = true)
    : BST lo hi → BST lo hi
  | .leaf ⇒ .node k p1 p2 .leaf .leaf
  | .node k' q1 q2 left right ⇒
    match h : N.ble k k' with
    | true  ⇒
      .node k' q1 q2
        (insert k p1 h left) right
    | false ⇒
      .node k' q1 q2
        left
        (insert k (ble_false_rev h) p2 right)
```

What the type checker verifies

In the `insert` function:

- The `leaf` case builds a new node. The proofs p_1 and p_2 witness that k fits within the bounds lo and hi. The type checker confirms this.
- Going left: the left subtree has type BST lo $(\text{val } k')$. We need $k \leq k'$ as the new upper-bound proof. The match equation `h : N.ble k k' = true` provides exactly this.
- Going right: symmetrically, we need $k' \leq k$. The match gives `h : N.ble k k' = false`, and the helper `ble_false_rev` derives `N.ble k' k = true` from it.

The type forces us to produce these proofs. We cannot forget a case.

Contrast with the unverified version

```
-- Unverified insert: no proof obligations
def Tree.insert (k : N) : Tree N → Tree N
  | .leaf ⇒ .node .leaf k .leaf
  | .node left k' right ⇒
    if N.ble k k' then
      .node (Tree.insert k left) k' right
    else
      .node left k' (Tree.insert k right)
```

This compiles no matter what. A bug in the comparison logic would produce an invalid tree, and no type error would catch it.

With BST, a bug in the comparison logic means the proof obligations fail, and the function does not typecheck. The invariant is enforced *at compile time*.

What `insert` does NOT guarantee

The type says `BST lo hi → BST lo hi`: the ordering invariant is preserved.
But this also typechecks:

```
def discard : BST lo hi → BST lo hi
  | _ ⇒ .leaf
```

The type says nothing about **contents**. We could prove correctness externally, but there is a better way: internalize the contents too.

Internalizing membership

Index the BST by a membership predicate mem : $N \to$ Prop:

```
inductive BSTM : Bound → Bound → (N → Prop) → Type where
  | leaf : BSTM lo hi (fun _ ⇒ False)
  | node : (k : N)
    → Bound.ble lo (.val k) = true
    → Bound.ble (.val k) hi = true
    → BSTM lo (.val k) memL
    → BSTM (.val k) hi memR
    → BSTM lo hi (fun x ⇒ memL x ∨ x = k ∨ memR x)
```

The third index is a *set*: $N \to$ Prop. A leaf contains nothing ($\bot$). A node with key k, left set memL, and right set memR contains memL $\cup \{k\} \cup$ memR.

What this gives us

Now the type of `insert` says *exactly* what insert does:

```
def BSTM.insert (k : N) (p1 p2)
    : BSTM lo hi mem
    → BSTM lo hi (fun x ⇒ x = k ∨ mem x)
```

The return type says: the output contains exactly the elements of the input, plus k. The type *is* the specification. If the function typechecks, it is correct.

Implementing BSTM.insert

```
-- Named cast: simp can see through this
def BSTM.castMem {P Q : N → Prop} (h : P = Q)
    : BSTM lo hi P → BSTM lo hi Q := h ▸ id

@[simp] theorem BSTM.search_castMem (h : P = Q)
    (k : N) (t : BSTM lo hi P)
    : BSTM.search k (BSTM.castMem h t)
    = BSTM.search k t := by subst h; rfl

def BSTM.insert (k : N) (p1 p2)
    : BSTM lo hi mem
    → BSTM lo hi (fun x ⇒ x = k ∨ mem x)
  | .leaf ⇒
    BSTM.castMem (by funext x; simp [or_comm, false_or])
      (.node k p1 p2 .leaf .leaf)
  | @BSTM.node _ _ memL memR k' q1 q2 left right ⇒
    match hc : N.ble k k' with
    | true  ⇒
      BSTM.castMem (by funext x; simp [or_assoc])
        (.node k' q1 q2 (BSTM.insert k p1 hc left) right)
```

The cost

The node constructor produces fun $x \Rightarrow$ memL $x \vee x = k \vee$ memR x. After inserting into the left subtree, the new set is $(x = k' \vee \text{memL } x) \vee x = k'' \vee \text{memR } x$. We need this to equal $x = k' \vee (\text{memL } x \vee x = k'' \vee \text{memR } x)$.

These are *propositionally* equal (associativity of $\vee$) but not *definitionally* equal. So we need a named cast (`castMem`) — the same issue as `vappend` with the wrong add.

Problem	**Vectors**	**BST membership**
Gap	add $0\ n \not\equiv n$	$(A \vee B) \vee C \not\equiv A \vee (B \vee C)$
Fix	`rw [zero_add]`	`castMem (by simp [or_assoc])`

The technique is the same: enter tactic mode and `rw` the goal. The pain is proportional to how far your computation diverges from definitional equality.

The design space for BSTs

Type	**Ordering**	**Contents**
Tree A	none	none
BST lo hi	internal	none
BST lo hi + theorems	internal	external
BSTM lo hi mem	internal	internal

Each step internalizes more. The last row is the most informative type — but also the heaviest to work with.

Section 5

Part V: Discussion

Internal vs. external: when to use which

	External	**Internal**
Invariant location	separate theorem	in the type
Can forget to verify	yes	no
Refactoring	update proofs separately	types enforce consistency
Complexity	simpler definitions	richer types
Flexibility	any property, any time	must design types upfront

Internal verification catches bugs earlier (at type-checking time) but requires more upfront design. External verification is more flexible but can be neglected.

The spectrum

In practice, you rarely use one style exclusively. A typical verified program mixes both:

- **Key invariants** (ordering, length, well-formedness) go in the type — these are the ones you cannot afford to forget.
- **Derived properties** (commutativity, associativity, specific performance bounds) are proved externally — they are important but do not need to be in every function signature.

The art is choosing which invariants to internalize. Too few, and bugs slip through. Too many, and the types become unwieldy.

What we built today

Structure	**Invariant**	**Verification**
List A	(none)	—
List A + `length_append`	length additive	external
Vec $A\ n$	length in type	internal
Tree A	(none)	—
Tree A + `is_bst` theorem	ordering	external
BST $lo\ hi$	ordering in type	internal

The pattern

The method for internal verification follows a pattern:

1. **Identify the invariant** you want to enforce (length, ordering, balance, ...).
2. **Add indices** to the type that capture the invariant.
3. **Constrain constructors** so that each one produces values satisfying the invariant.
4. **Carry proofs** in constructors when the invariant involves relationships between indices.

The result is a type where **every value is valid by construction**. The type checker acts as a continuous verification engine.

Exercises

1. Explain why the case $(\text{nil}, \text{cons}\ \ldots)$ is impossible in `vzip`. What role does the shared index n play?
2. Define vconcat : Vec $(\text{Vec}\ A\ m)\ n \to \text{Vec}\ A\ (\text{mul}\ n\ m)$. Which definition of `mul` avoids casts?
3. Write toList : Vec $A\ n \to \text{List}\ A$. Does it need any proofs?
4. Uncomment the BST node with $k = 3$, $lo = \text{val}\ 5$ in the Lean file. Which proof obligation fails?
5. (Harder) Define `SortedList.insert` that maintains the ordering invariant.
6. (Harder) Define `BSTM.search` and prove that after inserting k, searching for k succeeds.

All exercise stubs are in the companion Lean file as `sorry` placeholders.

Next time

Next lecture: type-level computation and more automation.

How Lean resolves notation and overloaded operations via type classes, the elaborator, and how tactics like `simp` and `omega` automate the kind of proofs we have been writing by hand.